WEEK 2 · DAY 3 · OVERVIEW

XGBoost, Regularization, and Early Stopping



Created by **Jeff Lenamon**

ABOUT 55 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Statement

BUSINESS CONTEXT

Day 7 built gradient boosting from residuals on bonds and named scikit-learn's version for a yes-or-no target.

The desk now runs XGBoost, the boosting library many teams reach for first, on Course 4's card accounts.

It needs two answers: how many trees to keep, and how to stop trees six questions deep from memorising 18,000 accounts.

The accounts are real, from Taiwan in 2005, so they describe consumer credit there and then, not US credit today.

The four person columns are never features, because a model that used them would sort people by them.

OBJECTIVES

Fit XGBoost and score it on training and validation rows, follow its scores tree by tree, and stop the trees on rows carved from the training rows.

Rebuild a leaf's value and a split's gain by hand, measure each brake on five folds inside the training rows, weight the rare class, and compare XGBoost with the course's earlier models.

ABOUT 55 MIN WITH EXERCISES / 11 SECTIONS / 4 EXERCISES

Your Route Through Day 3

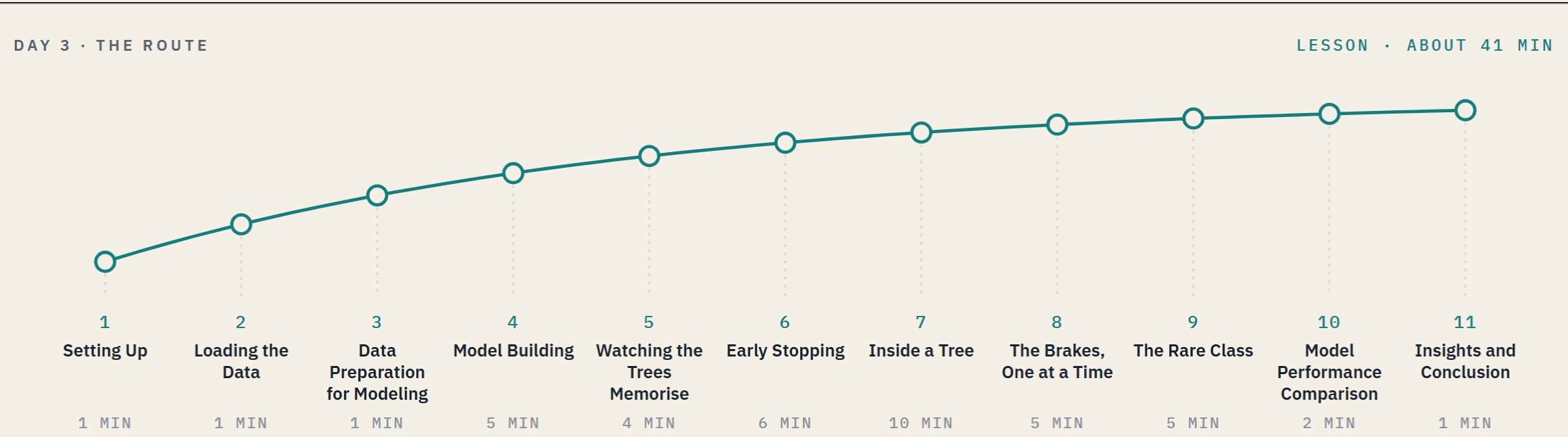


FIGURE 1 Eleven sections, in order, with about how long each takes.

Time. About 41 minutes for the lesson, then about 12 for the four exercises.

Fix the seed and load the libraries.

Setting Up

THE SETUP

```
import os
from pathlib import Path
os.environ["OMP_NUM_THREADS"] = "1"
os.environ["OPENBLAS_NUM_THREADS"] = "1"
SEED = 42
DATA = "../../../data/"
if not Path(DATA + "benchmark.csv").exists():
    DATA = "https://raw.githubusercontent.com/lenamonj/python-fixed-income/main/data/"
```

XGBOOST BESIDE THE EARLIER MODELS

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier
```

Read the card file and pick 19 columns.

Loading the Data

THE FILE'S COLUMNS

- **LIMIT_BAL**: credit limit, NT dollars
- **PAY_0**, **PAY_2** to **PAY_6**: repayment status, September back to April 2005, where -1 is paid on time, 1 to 9 is months late, and -2 and 0 are undocumented (used as numbers, a modelling choice, as in Course 4)
- **BILL_AMT1** to **BILL_AMT6**: the bill, NT dollars, September back to April
- **PAY_AMT1** to **PAY_AMT6**: the amount paid, NT dollars, September back to April
- **default payment next month**: 1 if the account defaulted the following month, **the target**
- **ID**, **SEX**, **EDUCATION**, **MARRIAGE**, **AGE**: never used

READING THE CARD FILE

```
card = pd.read_csv(DATA + "credit_card_default.csv")
TARGET = "default payment next month"
FEATURES = list(X.columns)
```

ROWS AND THE DEFAULT SHARE

```
print(card.shape)
print("Defaults:", y.sum(), " Share:", round(y.mean() * 100, 2), "percent")
```

```
(30000, 25)
Defaults: 6636  Share: 22.12 percent
```

Rebuild Course 4's training, validation and test rows.

Data Preparation for Modeling

COURSE 4'S SPLIT

```
X_rest, X_test, y_rest, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEED)
X_train, X_valid, y_train, y_valid = train_test_split(X_rest, y_rest, test_size=0.25, stratify=y_rest, random_state=SEED)
print("Training:", len(X_train), " Validation:", len(X_valid), " Test:", len(X_test))
print("Default share:", round(y_train.mean(), 4), round(y_valid.mean(), 4), round(y_test.mean(), 4))
```

```
Training: 18000  Validation: 6000  Test: 6000
Default share: 0.2212 0.2212 0.2212
```

THE THREE KINDS OF ROWS

The same accounts as Course 4 and week 1, each set with the same default share.

Every score below is on the validation rows; Course 5 chooses nothing on them and never scores the test rows.

Training rows fit, validation rows report, test rows stay closed.

Fit XGBoost's defaults and score both row sets.

Model Building

XGBOOST IN THREE LINES

XGBoost is Day 7's gradient boosting run on the log-odds of default, Course 4's Day 6 scale.

THE MODEL

```
xgb = XGBClassifier(n_jobs=1, random_state=SEED).fit(X_train, y_train)
```

THE SETTINGS IT CHOSE

```
print({name: params[name] for name in ["n_estimators", "learning_rate", "max_depth", "reg_lambda"]})
```

```
{'n_estimators': None, 'learning_rate': None, 'max_depth': None, 'reg_lambda': None}
```

EVERY NODE OF THE 100 TREES

```
print("Trees:", len(leaves), " Leaves per tree: mean", round(leaves.mean(), 1), " fewest", leaves.min(), " most", leaves.max())
```

```
Trees: 100  Leaves per tree: mean 42.3  fewest 12  most 63
```

Score every tree count up to 300.

Watching the Trees Memorise

THE QUESTION

`predict_proba(rows, iteration_range=(0, k))` scores the rows with the first k trees only, so one model of 300 trees gives the scores of 300 models.

Q. Sketch ROC AUC on the training rows and on the validation rows against the number of trees, from 1 to 300.

ONE MODEL, 300 SCORES

```
xgb300 = XGBClassifier(n_estimators=300, n_jobs=1, random_state=SEED).fit(X_train, y_train)
for k in range(1, 301):
    train_out = xgb300.predict_proba(X_train, iteration_range=(0, k))[:, 1]
    valid_out = xgb300.predict_proba(X_valid, iteration_range=(0, k))[:, 1]
```

Early Stopping

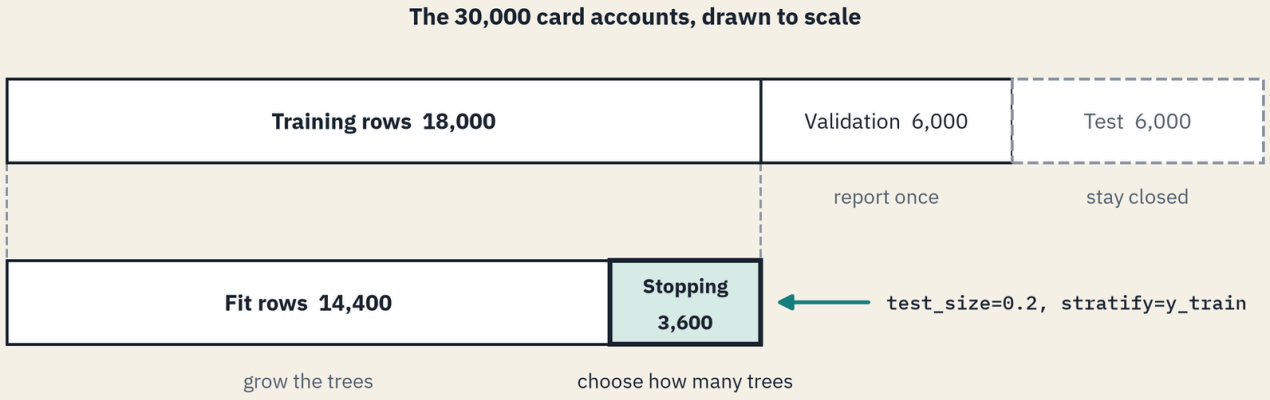


FIGURE 2 The 18,000 training rows cut into 14,400 fit rows and 3,600 stopping rows; validation reports, test stays closed.

EARLY STOPPING

Early stopping grows trees one at a time and stops once a score on rows the trees never fit stops improving.

Those rows are the **stopping rows**, cut from the training rows, so the validation rows only report.

CARVING THE STOPPING ROWS

```
X_fit, X_stop, y_fit, y_stop = train_test_split(X_train, y_train, test_size=0.2, stratify=y_train, random_state=SEED)
print("Fit rows:", len(X_fit), " Stopping rows:", len(X_stop), " Default share:", round(y_fit.mean(), 4), round(y_stop.mean(), 4))
print("Stopping rows that are also validation rows:", len(X_stop.index.intersection(X_valid.index)))
```

```
Fit rows: 14400  Stopping rows: 3600  Default share: 0.2212 0.2211
Stopping rows that are also validation rows: 0
```

Inside a Tree

One leaf at round one: every account starts at p = 0.2212

account	y	gradient p - y	hessian p(1 - p)
defaulted	1	-0.7788	0.1723
defaulted	1	-0.7788	0.1723
defaulted	1	-0.7788	0.1723
paid	0	0.2212	0.1723
sums		**G = -2.1151**	**H = 0.6891**

leaf value

`-step x G / (H + lambda)`

`-0.3 x -2.1151 / (0.6891 + 1)`

0.3757

log-odds added

lambda adds to H: a leaf with few accounts has a small H, so lambda shrinks it most

FIGURE 3 One leaf of three defaulted accounts and one paid account: G -2.1151, H 0.6891, lambda 1, a value of 0.3757 in log-odds.

THE LEAF'S VALUE

On the log-odds, each account's **gradient** is p - y, the slope of its log loss, and its **hessian** is p(1 - p), its curvature.

A leaf's value is -step x G / (H + lambda), where G and H are the leaf's gradients and hessians summed and lambda is **`reg_lambda`**.

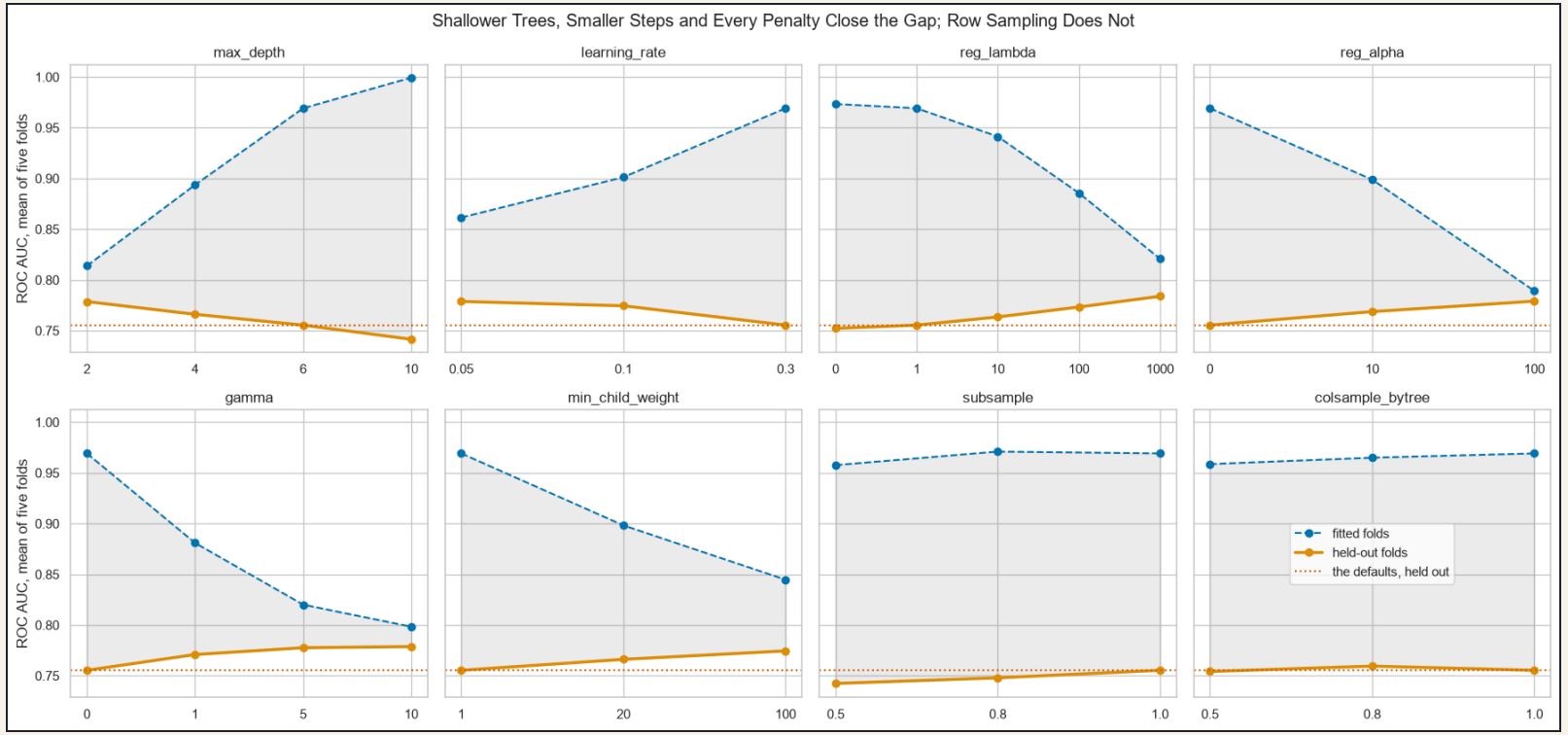
EACH LEAF BY HAND

```
print("Accounts:", left.sum(), (~left).sum(), " G:", round(G_left, 2), round(G_right, 2), " H:", round(H_left, 2), round(H_right, 2))
leaf_left, leaf_right = -0.3 * G_left / (H_left + 1), -0.3 * G_right / (H_right + 1)
print("Leaf values by hand:", round(leaf_left, 4), round(leaf_right, 4))
```

```
Accounts: 16115 1885  G: 905.0 -905.0  H: 2776.34 324.75
Leaf values by hand: -0.0978 0.8334
```

Score each brake on five training folds.

The Brakes, One at a Time



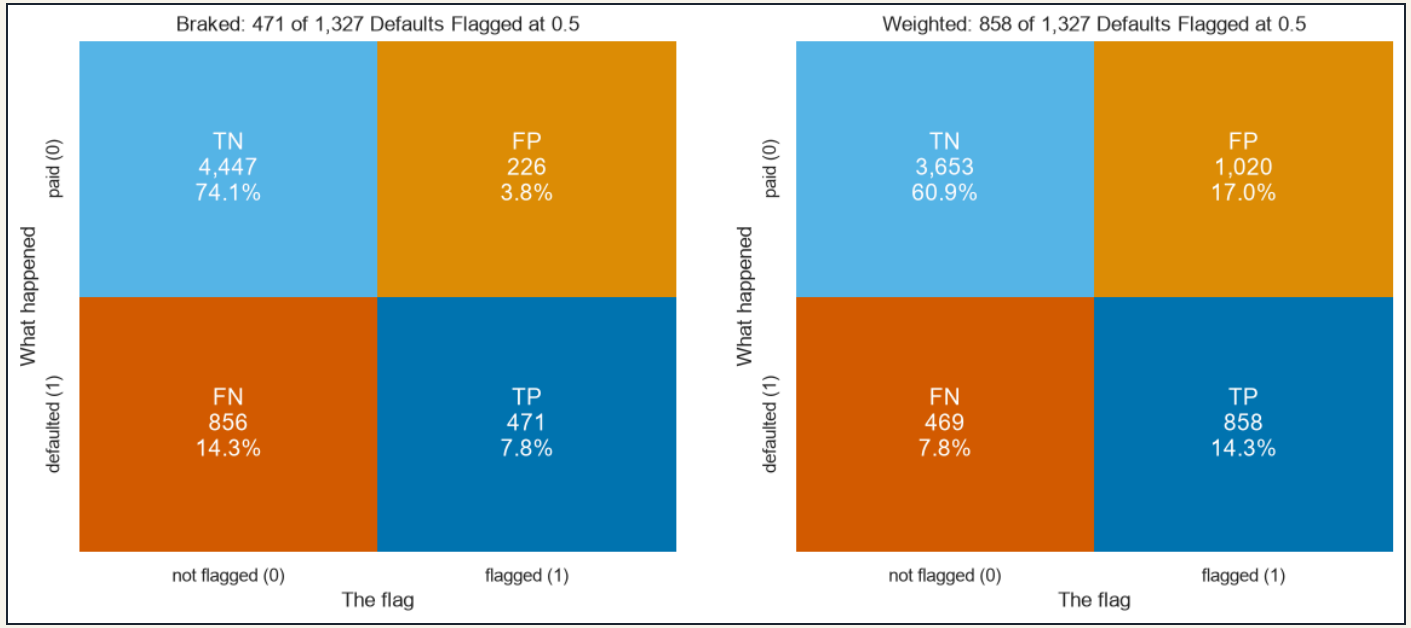
THE QUESTION

A tree whose every question gains less than gamma is cut back to its root, a single leaf, which adds the same number to every account.

Q. With **gamma=5**, how many of 300 trees end as a single leaf? Write down a number.

Brakes let many small steps learn slowly, and early stopping picks how many.

The Rare Class



THE WEIGHT

scale_pos_weight multiplies every defaulted account's gradient and hessian, so each counts that many times in every leaf.

THE WEIGHTED TWIN

```
twin = XGBClassifier(n_estimators=braked.best_iteration + 1, learning_rate=0.05, reg_lambda=1000, scale_pos_weight=weight,
                     n_jobs=1, random_state=SEED).fit(X_fit, y_fit)
twin_out = twin.predict_proba(X_valid)[:, 1]
print("Average output: braked", round(braked_out.mean(), 4), " weighted", round(twin_out.mean(), 4))
print("ROC AUC: braked", round(roc_auc_score(y_valid, braked_out), 4), " weighted", round(roc_auc_score(y_valid, twin_out), 4))
```

Model Performance Comparison

TEN MODELS

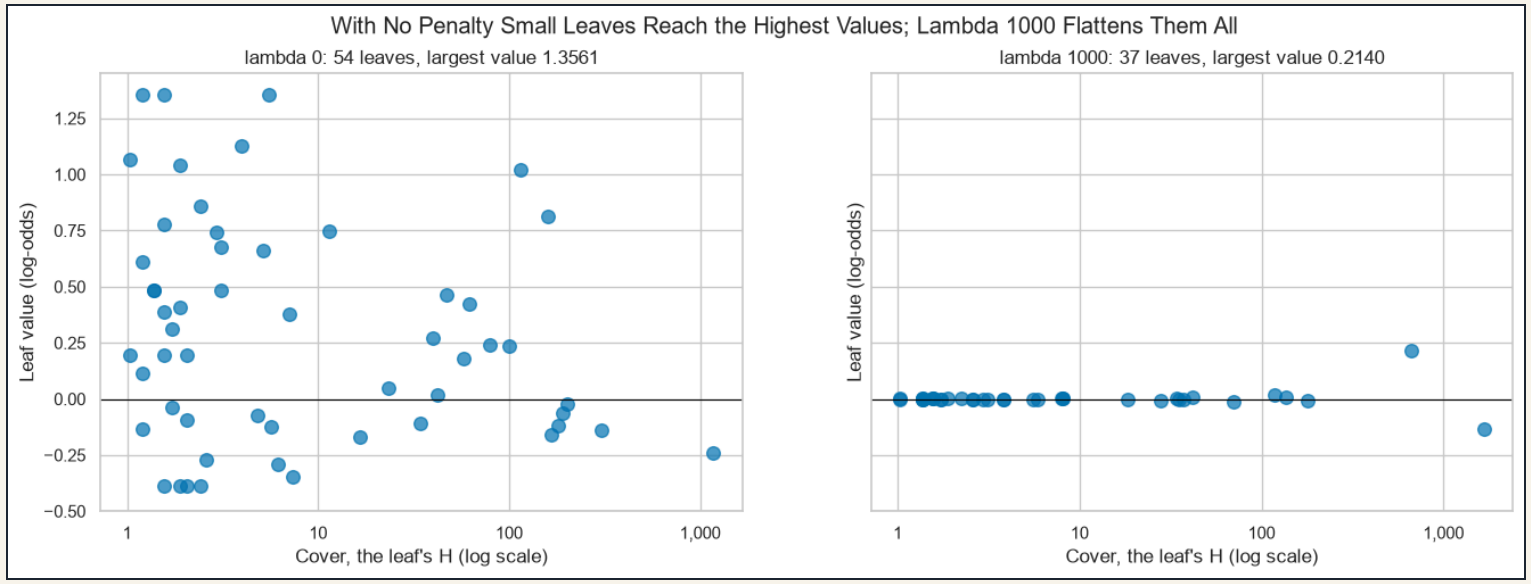
```
models = {"Logistic regression (Course 4)": Pipeline([("scale", StandardScaler()), ("logit", LogisticRegression(max_iter=1000))]),
          "Pruned tree (Course 4)": DecisionTreeClassifier(class_weight="balanced", ccp_alpha=0.001, random_state=SEED),
          "Chosen forest (Day 3)": RandomForestClassifier(n_estimators=200, max_features=8, min_samples_leaf=100,
                                                          random_state=SEED, n_jobs=1),
          "AdaBoost, 200 stumps (Day 6)": AdaBoostClassifier(n_estimators=200, random_state=SEED),
          "Gradient boosting defaults (Day 7)": GradientBoostingClassifier(random_state=SEED),
          "XGBoost defaults": xgb, "XGBoost stopped at 0.3": stop_03, "XGBoost stopped at 0.05": stop_005,
          "XGBoost braked": braked, "XGBoost weighted": twin}
for name in list(models)[:5]:
    models[name].fit(X_train, y_train)
```

READING THE TABLE

Recall and precision at 0.5 are not comparable across rows: the pruned tree and the twin were weighted toward defaults.

Braked and stopped, XGBoost ranks the validation accounts within a few thousandths of Day 3's chosen forest; Day 10 asks how much of such a gap is noise.

Insights and Conclusion



ONE PANEL

```
def leaf_panel(lam, ax):
    table = first_leaves[lam]
    ax.scatter(table["Cover"], table["Gain"], s=70, alpha=0.7)
```

Day 9 grows trees leaf by leaf with LightGBM and stacks models.

You Can Now

- Fit **`XGBClassifier`** on one thread and score it on training and validation rows.
- Follow a model's scores tree by tree with **`iteration_range`**.
- Carve stopping rows from the training rows and stop the trees on them with **`early_stopping_rounds`**.
- Rebuild a leaf's value and a split's gain from gradients and hessians, and read lambda and gamma in them.
- Compare brakes on folds inside the training rows, and weight the rare class with **`scale_pos_weight`**.

NEXT
WEEK 2 · DAY 4

LightGBM and Stacking

Day 9 grows trees leaf by leaf with LightGBM and stacks models.